

EJERCICIOS DE INTRODUCCIÓN A PYTHON

Algoritmos y Estructuras de Datos

Objetivo. Practicar contenidos de Python básico: variables, tipos, operadores y funciones; colecciones y control de flujo; y errores, excepciones y validación de entradas. Los ejercicios no requieren programación orientada a objetos.

Ejercicio 6. Ficha de un alumno

Vamos a registrar el nombre y la nota de un alumno. El programa debe comprobar que la nota tiene sentido y después clasificarla.

Qué tienes que hacer

1. Pide el nombre del alumno.
2. Pide su nota. Debe ser un número entre 0 y 10.
3. Si el usuario escribe texto o un número fuera del rango 0–10, vuelve a pedir la nota.
4. Clasifica la nota: menos de 5 → Suspenso; de 5 a menos de 7 → Aprobado; de 7 a menos de 9 → Notable; de 9 a 10 → Sobresaliente.
5. Guarda nombre y nota en un diccionario con las claves nombre y nota.
6. Muestra una frase como: “Ana ha obtenido un 8.2 → Notable”.

Conceptos principales que debes utilizar: diccionarios · while · try / except ValueError · if / elif / else · input() · float() · f-strings

Comprobación rápida: Una nota de 7.0 debe clasificarse como Notable.

Ejercicio 7. Analizar un grupo de alumnos

Ahora ya no tenemos un solo alumno, sino un grupo representado mediante una lista de diccionarios.

Qué tienes que hacer

1. Parte de la lista de alumnos indicada en el enunciado.
2. Muestra el nombre y la nota de todos los alumnos.
3. Calcula la nota media del grupo.
4. Crea una lista con los nombres de los alumnos aprobados (nota ≥ 5).
5. Averigua qué alumno tiene la nota más alta.
6. Muestra un resumen final con media, aprobados y mejor nota.

Datos de partida: `alumnos = [{"nombre": "Ana", "nota": 8.5}, {"nombre": "Luis", "nota": 4.2}, {"nombre": "María", "nota": 9.1}, {"nombre": "Pablo", "nota": 6.0}, {"nombre": "Elena", "nota": 5.5}]`

Conceptos principales que debes utilizar: lista de diccionarios · for · acumuladores · if · append() · acceso por clave

Comprobación rápida: Usa estos datos: Ana 8.5, Luis 4.2, María 9.1, Pablo 6.0 y Elena 5.5.

Ejercicio 8. Buscador de productos y precios

Una pequeña tienda guarda sus productos en un diccionario. Queremos que el usuario pueda consultar precios sin que el programa falle si escribe un producto inexistente.

Qué tienes que hacer

1. Crea el diccionario: `precios = {"pan": 1.20, "leche": 1.05, "café": 3.50, "arroz": 1.80, "huevos": 2.40}`.
2. Pide al usuario el nombre de un producto.
3. Normaliza el texto con `lower()` para que “PAN” y “pan” se traten igual.
4. Si el producto existe, muestra su precio.
5. Si no existe, muestra un mensaje indicando que no está disponible.
6. Después, muestra la lista de productos disponibles.
7. Resuelve la búsqueda evitando un `KeyError`. Puedes usar `in` o `get()`.

Conceptos principales que debes utilizar: diccionarios · `in` o `get()` · `KeyError` y prevención · `str.lower()` · for · `input()` · f-strings

Comprobación rápida: Si el usuario escribe CAFÉ, debe encontrar el producto café.

Ejercicio 9. Funciones para gestionar alumnos

Vamos a reorganizar operaciones habituales sobre alumnos para no repetir código. Cada operación tendrá su propia función.

Qué tienes que hacer

1. Usa una lista de diccionarios con varios alumnos y sus notas.
2. Crea `es_aprobado(nota)`, que devuelva `True` si la nota es al menos 5.
3. Crea `calcular_media(alumnos)`, que devuelva la media de todas las notas.
4. Crea `buscar_alumno(nombre, alumnos)`, que devuelva el diccionario del alumno si lo encuentra y `None` si no existe.
5. Haz que `buscar_alumno` no distinga entre mayúsculas y minúsculas.

6. Prueba las tres funciones con assert.

7. Muestra un ejemplo de uso de cada función.

Conceptos principales que debes utilizar: funciones · parámetros · return · booleanos · listas de diccionarios · for · lower() · None · assert

Comprobación rápida: buscar_alumno("ana", alumnos) debe encontrar también a una alumna guardada como "Ana".

Ejercicio 10. Mini sistema de gestión de alumnos

Este ejercicio reúne las piezas anteriores en un programa con menú. En este momento si no quieres no necesitas clases ni programación orientada a objetos: solo listas, diccionarios, funciones, bucles y control de errores.

Qué tienes que hacer

1. Empieza con una lista vacía llamada alumnos.
2. Muestra repetidamente un menú: 1) Añadir alumno, 2) Mostrar alumnos, 3) Mostrar aprobados, 4) Calcular media, 5) Salir.
3. Al añadir un alumno, pide nombre y nota. La nota debe ser numérica y estar entre 0 y 10.
4. Guarda cada alumno como diccionario dentro de la lista.
5. La opción Mostrar alumnos debe listar nombre y nota de todos los alumnos.
6. La opción Mostrar aprobados debe enseñar solo quienes tengan nota ≥ 5 .
7. La opción Calcular media debe mostrar la media. Si todavía no hay alumnos, debe avisarlo en lugar de dividir entre cero.
8. El menú debe seguir apareciendo hasta que el usuario elija Salir.
9. Organiza las tareas principales en funciones.

Conceptos principales que debes utilizar: while · menú · listas de diccionarios · funciones · if / elif / else · try / except · validación · append() · for

Comprobación rápida: Después de añadir Ana 8 y Luis 4, "Mostrar aprobados" debe enseñar solo a Ana.

Indicaciones generales de entrega

- Entrega el código en formato .ipynb o .py según se indique en clase.
- El programa debe ejecutarse sin errores con los casos normales indicados en el enunciado.
- Usa nombres de variables y funciones que permitan entender qué representa cada dato.
- Añade comentarios cuando ayuden a explicar una decisión, pero no hace falta comentar cada línea.
- No se requiere POO, clases, librerías externas, pandas ni NumPy para ninguno de estos ejercicios.
- Cuando se solicite control de errores, evita except genéricos y captura la excepción concreta esperada.
- Prueba el código con más de un caso antes de entregarlo.